

Why Python Developers Should Care About Asynchronous Programming

Explore what asynchronous programming is, when it should be used, how Python supports it, and measure its performance as compared to synchronous programming.



In the era of data-driven applications and network-intensive systems, writing efficient, scalable Python code is more important than ever. When working with tasks like downloading files, calling APIs, or reading from databases, performance can take a hit if your program spends most of its time waiting.

This is where asynchronous programming shines. It is a way of writing code where tasks that take

time — like file downloads or network requests — don't block the entire program while they finish. Instead, your program continues doing other things.

This model is especially powerful for I/O-bound tasks, where your CPU spends time waiting for external responses (like servers or disks), rather than doing computations.

When should you use it?

Asynchronous programming is most

beneficial when:

- Your application performs many network or disk I/O operations.
- You're working with concurrent user requests (e.g., in a web server).
- You want to optimise waiting time in data pipelines or API calls.
- You're building apps that need to scale to thousands of tasks concurrently.

However, if your tasks are CPU-heavy (like model training or image processing), multithreading or multiprocessing is more appropriate.

How Python supports asynchronous programming

Python provides native support for async workflows using:

- `async` and `await` keywords to define asynchronous functions (coroutines).
- `asyncio`: The standard library for running async tasks and managing the event loop.
- `aiohttp`: A non-blocking HTTP client/server built on top of `asyncio`.

Use case: Downloading a 10MB file 10 times

To demonstrate the efficiency of asynchronous programming, we conducted an experiment where a 10MB file was downloaded from the internet ten times—first using sequential code and then using asynchronous code, as illustrated in Figure 1. The time taken for each